

Kitchen Editor Implementation Guideline — SceneEntity Parent Refactor

This guideline reflects the current refactor direction where `SceneEntity` is the parent model for movable/selectable design objects. `PlacedAssembly` and `DesignReservationZone` are child scene entities. Shared behavior belongs to `SceneEntity`; only explicit child-specific behavior stays in the child domain.

1. Source-of-truth scene model

`DesignScene` is the runtime source of truth.

Current target shape:

```
export type DesignScene = Readonly<{
  sceneEntities: readonly SceneEntity[];
  placedWallGraphs: readonly PlacedWallGraph[];
  activeSelection: SceneSelection | null;
  activeSceneOperation: SceneOperation | null;
}>;
```

Do not reintroduce separate live scene arrays such as:

```
placedAssemblies
designReservationZones
```

Those objects must be read from `sceneEntities` through scene-entity collection readers.

Allowed collection readers include:

```
getPlacedAssembliesFromSceneEntities(...)
getDesignReservationZonesFromSceneEntities(...)
getSceneEntityByRef(...)
getSceneEntitiesByRefs(...)
```

These are real source-of-truth accessors, not compatibility wrappers.

2. SceneEntity parent model

`SceneEntity` is the parent model for selectable/movable design objects.

```

export type SceneEntityKind =
  | "placed-assembly"
  | "design-reservation-zone";

export type SceneEntityBase<TKind extends SceneEntityKind> = Readonly<{
  id: string;
  entityKind: TKind;
  worldPositionInches: Point3DInches;
  rotationDegrees: Readonly<{
    zDegrees: number;
  }>;
}>;

export type SceneEntity =
  | PlacedAssembly
  | DesignReservationZone;

```

`worldPositionInches` always means:

center point of the scene-entity volume
`zInches` is vertical center, not bottom

Do not store:

`baseCenterPointInches`

Bottom/floor position is derived:

```

const distanceFromFloorInches =
  sceneEntity.worldPositionInches.zInches -
  getSceneEntitySizeInches(sceneEntity).heightInches / 2;

```

3. PlacedAssembly child responsibilities

Keep the existing name:

`PlacedAssembly`

Do not rename it to `AssemblyObject`.

`PlacedAssembly` owns only assembly-specific behavior:

```
definitionId
configuration
catalog selection
assembly dimension controls through configuration
assembly option groups
assembly tree building
primitive assembly rendering
front outline rendering
countertop cutout behavior
wall cutout behavior
placed assembly candidate factory
```

`PlacedAssembly` must not own generic scene-entity behavior such as:

```
position editing
rotation editing
selected highlight
placement lifecycle
move lifecycle
multi-drag
alignment
alignment guides
delete
duplicate
group behavior
wall measurement guides
```

4. DesignReservationZone child responsibilities

`DesignReservationZone` owns only reservation-zone-specific behavior:

```
reservedFor
default purpose
default dimensions
size editing
zone visual rendering
reservation-zone candidate factory
```

`DesignReservationZone` must not own generic scene-entity behavior such as:

```
baseCenterPointInches
position editing
rotation editing
```

```
selected highlight
placement lifecycle
move lifecycle
multi-drag
alignment
alignment guides
delete
duplicate
group behavior
wall measurement guides
```

Reservation zones use `worldPositionInches`, not `baseCenterPointInches`.

5. Parent-owned SceneEntity behavior

The following behavior belongs to `SceneEntity` ownership:

```
identity and refs
transform access
transform mutation
size access
distance-from-floor derivation
bounds
footprints
selected highlight
floor-plan edit controls
rotation controls
placement lifecycle
move lifecycle
rotation lifecycle
alignment
alignment guides
group selection
group bounds
group guides
group movement
delete
duplicate
wall measurement guides
shared transform property editing
history for scene-entity actions
keyboard delete/cancel behavior for scene entities
```

Do not duplicate these responsibilities in `PlacedAssembly` and `DesignReservationZone`.

6. SceneEntity refs and selection

Use a parent scene-entity ref:

```
export type SceneEntityRef = Readonly<{
  entityKind: SceneEntityKind;
  entityId: string;
}>;
```

Selection shape:

```
export type SceneSelection =
  | Readonly<{ kind: "scene-entity"; sceneEntity: SceneEntityRef }>
  | Readonly<{ kind: "scene-entities"; sceneEntities: readonly
SceneEntityRef[] }>
  | Readonly<{ kind: "placed-wall-segment"; wallGraphId: string; wallSegmentId:
string }>;
```

Wall segments are not scene entities. Keep wall graph/topology behavior separate.

7. SceneEntity transform ownership

Scene-entity transform behavior belongs in:

```
src/engine/scene-entities/sceneEntityTransforms.ts
```

This file owns:

```
getSceneEntitySizeInches(...)
getSceneEntityDistanceFromFloorInches(...)
createSceneEntityWithWorldPosition(...)
createSceneEntityWithRotationZ(...)
createSceneEntityWithDistanceFromFloor(...)
createSceneEntityWithHeightPreservingDistanceFromFloor(...)
```

Size access differs by child:

```
PlacedAssembly -> configuration.sizeInches
DesignReservationZone -> sizeInches
```

Callers should use the parent accessor and not branch manually unless they are implementing child-specific behavior.

8. SceneEntity collection editing

Scene-entity collection editing belongs in:

```
src/engine/scene-entities/sceneEntityCollectionEditing.ts
```

This file owns:

```
getPlacedAssembliesFromSceneEntities(...)  
getDesignReservationZonesFromSceneEntities(...)  
getSceneEntityByRef(...)  
getSceneEntitiesByRefs(...)  
replaceSceneEntity(...)  
replaceSceneEntities(...)  
removeSceneEntities(...)  
duplicateSceneEntities(...)
```

Avoid repeated local map/filter mutation logic inside action modules.

9. Bounds and footprint ownership

Parent bounds entry:

```
src/engine/scene-entities/sceneEntityBounds.ts
```

Child-specific bounds builders may remain focused:

```
src/engine/scene-entities/placedAssemblySceneEntityBounds.ts  
src/engine/scene-entities/designReservationZoneSceneEntityBounds.ts
```

Shared callers should use:

```
createSceneEntityBounds(...)  
createSceneEntityBoundsForRefs(...)
```

`SceneEntityBounds` should not expose `baseCenterPointInches`.

Use:

```
centerPointInches
sizeInches
rotationDegrees
footprint
footprintCornersInches
topCornersInches
heightRangeInches: {
  minZInches
  maxZInches
}
```

10. Selected highlight ownership

Selected scene-entity UI belongs under:

```
src/features/kitchen-editor/rendering/scene-entities/
SelectedSceneEntityLayer.tsx
```

Parent-owned selected visuals include:

```
SceneEntityVolumeBoundingBox
SceneEntityFloorPlanEditControls
SceneEntityFloorPlanRotationControl
SceneEntityGroupGuides
SceneEntityWallMeasurementGuides
```

Do not add assembly-only or reservation-zone-only selected highlight layers.

Child renderers may own click targets, but they must call the same parent selection action.

11. SceneEntity placement lifecycle

Use one placement operation:

```
scene-entity-placement
```

Do not reintroduce:

```
assembly-placement  
design-reservation-zone-placement
```

Parent-owned placement actions belong in:

```
src/engine/scene/actions/sceneEntityPlacementActions.ts
```

Parent-owned placement surface belongs in:

```
src/features/kitchen-editor/interaction/scene-entities/  
SceneEntityPlacementSurface.tsx
```

Child-specific candidate factories remain separate:

```
PlacedAssembly candidate = catalog/configuration  
DesignReservationZone candidate = toolbar/default purpose/default dimensions
```

Once a candidate exists, the parent owns:

```
waiting-for-pointer  
pointer update  
alignment  
candidate positioning  
commit  
cancel  
selection after commit  
history entry
```

12. SceneEntity move lifecycle

Use one move operation:

```
scene-entity-move
```

Do not reintroduce:


```
assembly-move  
design-reservation-zone-move  
scene-entity-multi-move
```

Parent move actions belong in:

```
src/engine/scene/actions/sceneEntityMoveActions.ts
```

Parent move surface belongs in:

```
src/features/kitchen-editor/interaction/scene-entities/  
SceneEntityMoveDragSurface.tsx
```

Single-object drag and multi-object drag use the same state/action. A single drag is a one-item scene-entity move.

Move geometry belongs in:

```
src/engine/scene-entities/sceneEntityMoveGeometry.ts
```

It should preserve:

```
floor-plan movement behavior  
elevation movement behavior  
wall-face elevation drag frame behavior  
bottom-at-floor clamp  
mixed scene-entity group movement  
history/cancel/finish behavior
```

13. SceneEntity rotation lifecycle

Use one rotation operation:

```
scene-entity-rotation
```

Do not reintroduce:

```
assembly-rotation  
design-reservation-zone-rotation
```

Parent rotation actions belong in:

```
src/engine/scene/actions/sceneEntityRotationActions.ts
```

Parent rotation surface belongs in:

```
src/features/kitchen-editor/interaction/scene-entities/  
SceneEntityRotationSurface.tsx
```

Rotation is floor-plan-only.

Rotation state should include:

```
startWorldPositionInches  
startRotationDegrees  
latestValidRotationDegrees  
startPointerAngleDegrees  
startHandleCenterAngleDegrees  
latestHandleCenterAngleDegrees
```

Rotation should use the parent snapping file:

```
src/engine/scene-entities/sceneEntityRotationSnapping.ts
```

The rotation surface should finish on window pointer-up so rotation does not get stuck when the pointer is released outside the surface.

14. SceneEntity alignment ownership

Alignment belongs to `SceneEntity`, not assembly or reservation-zone ownership.

Alignment files belong under:

```
src/engine/scene-entities/alignment/
```

Use focused responsibility files:

```
sceneEntityAlignmentConstants.ts
sceneEntityAlignmentTypes.ts

sceneEntityPlanAlignmentTargets.ts
sceneEntityPlanAlignmentCandidates.ts
sceneEntityPlanAlignmentGuides.ts

sceneEntityElevationAlignmentBoxes.ts
sceneEntityElevationAlignmentCandidates.ts
sceneEntityElevationAlignmentGuides.ts

sceneEntityObjectAlignment.ts
```

This split is intentional. These are real behavior responsibilities, not fake helper files.

Supported alignment target kinds:

```
export type SceneEntityAlignmentTargetKind =
  | "scene-entity"
  | "wall-face"
  | "wall-centerline"
  | "floor-line";
```

Do not add derived countertop-opening or wall-opening alignment targets.

Sink/cooktop/window/door alignment happens through their `PlacedAssembly` scene entities.

15. Alignment guide behavior

Alignment guides are parent-owned.

Use:

```
SceneEntityAlignmentGuide
SceneEntityAlignmentGuides.tsx
activeSceneEntityAlignmentGuides
```

Do not reintroduce:

```
AssemblyObjectAlignmentGuide  
AssemblyObjectAlignmentGuides.tsx  
activeObjectAlignmentGuides
```

Renderer location:

```
src/features/kitchen-editor/rendering/scene-entities/  
SceneEntityAlignmentGuides.tsx
```

Plan guide rendering should use plan/world coordinates correctly.

Elevation guide rendering must use the elevation frame. Do not blindly render elevation guides as `[x, z, y]`.

16. Wall-face guide policy

Wall-face alignment should be careful.

Preferred behavior:

```
wall-face guide = visual guide first  
wall-face snap = only if explicitly implemented and tested
```

For elevation wall-face left/right edges:

```
show guide when object left/center/right is near wall-face left/center/right  
do not force object to snap to wall edge unless the feature is explicitly  
requested
```

This keeps alignment helpful without making wall snapping feel too aggressive.

17. Group behavior

Group behavior is parent-owned.

This includes:

```
mixed scene-entity selection  
group bounds  
group footprint  
group guide
```

```
group measurement
group drag
group alignment guides
group duplicate
group delete
group undo/redo
```

A mixed group may contain:

```
PlacedAssembly + PlacedAssembly
DesignReservationZone + DesignReservationZone
PlacedAssembly + DesignReservationZone
```

Do not show a single-object property editor for mixed or multi scene-entity selection.

18. Shared transform property UI

Shared transform editing belongs in:

```
src/engine/scene/actions/sceneEntityTransformEditingActions.ts
src/features/kitchen-editor/properties-panel/scene-entities/
SceneEntityTransformSection.tsx
```

The shared transform section owns:

```
X position
Y position
Distance from floor
Rotation
```

Assembly panel keeps:

```
dimensions
options
configuration-specific fields
```

Reservation-zone panel keeps:

```
reservedFor
width
```

```
depth
height
```

19. Delete and duplicate

Delete and duplicate remain scene-entity behavior.

Parent actions belong in:

```
src/engine/scene/actions/sceneEntityEditingActions.ts
```

Shared actions:

```
deleteSelectedSceneEntities(...)
duplicateSelectedSceneEntities(...)
```

Duplicate behavior is parent-owned, but duplicated child data must remain child-correct:

```
PlacedAssembly duplicate keeps definitionId/configuration
DesignReservationZone duplicate keeps reservedFor/sizeInches
```

20. Assembly placement feedback and validation

Do not reintroduce:

```
AssemblyPlacementFeedback
AssemblyPlacementInvalidReason
AssemblyPlacementResult
applyAssemblyPlacementRules
createAssemblyPlacementFeedback
assemblyPlacementFeedback
AssemblyPlacementFeedbackLayer
AssemblyPlacementBoundingBox
```

The current parent-owned placement flow is:

```
proposed world position
-> scene-entity alignment
-> update candidate/entity
```

No `isValid`, no `invalidReason`, no `latestValidWorldPositionInches`.

If future collision or placement validation is needed, design it deliberately as a new scene-entity or child-specific validation feature.

21. Rendering ownership

Child-specific visual renderers remain separate.

Assembly rendering owns:

```
AssemblyLayer  
PlacedAssemblyRenderer  
AssemblyRenderer  
AssemblyPrimitiveMesh  
AssemblyFrontOutlineLines  
useAssemblyRenderItems
```

Reservation-zone rendering owns:

```
DesignReservationZoneLayer  
DesignReservationZoneRenderer  
DesignReservationZonePlacementCandidateRenderer
```

Scene-entity rendering owns:

```
SelectedSceneEntityLayer  
SceneEntityVolumeBoundingBox  
SceneEntityFloorPlanEditControls  
SceneEntityFloorPlanRotationControl  
SceneEntityGroupGuides  
SceneEntityWallMeasurementGuides  
SceneEntityAlignmentGuides
```

Do not move selected highlight, duplicate/delete controls, group guides, or generic rotation controls back under assembly or reservation-zone rendering.

22. Interaction ownership

Parent interaction surfaces:

```
SceneEntityPlacementSurface  
SceneEntityMoveDragSurface  
SceneEntityRotationSurface
```

Do not reintroduce:

```
AssemblyDragSurface  
DesignReservationZoneDragSurface  
SceneEntityMultiDragSurface  
DesignReservationZonePlacementSurface
```

Pointer workflows for placement, movement, and rotation should be parent-owned. Child renderers can still provide child-specific visual geometry and click targets.

23. Derived cutout behavior

Cutout behavior remains `PlacedAssembly`-specific.

Derived countertop openings and wall openings should read placed assemblies from `sceneEntities`:

```
getPlacedAssembliesFromSceneEntities(sceneEntities)
```

Do not move cutout behavior to generic `SceneEntity`.

Do not align to derived countertop openings or derived wall openings. Align to the source scene entities instead.

24. Wall system boundary

Walls are not scene entities.

Keep wall-specific behavior separate:

```
wall graph topology  
wall segment splitting/merging  
wall segment selection  
wall elevation navigation  
wall face side settings  
wall geometry  
wall openings  
wall measurements
```


Scene entities may align to wall faces, wall centerlines, and floor line guides, but wall segments remain their own editing family.

25. History behavior

Scene-entity actions should record history for meaningful committed changes:

```
Place scene entity
Move scene entity
Rotate scene entity
Delete scene entity
Duplicate scene entity
Update scene entity transform
```

Use user-readable labels:

```
Place assembly
Place reservation zone
Move selected objects
Rotate assembly
Rotate reservation zone
Delete selected objects
Duplicate selected objects
```

Do not record history for transient hover, pointer preview, camera state, or alignment-guide-only updates.

26. Zustand selector rules

Subscribe to stable source values first.

Avoid selectors that return newly-created arrays or objects.

Preferred pattern:

```
const sceneEntities = useDesignSceneStore((state) =>
state.designScene.sceneEntities);
const activeSelection = useDesignSceneStore((state) =>
state.designScene.activeSelection);

const selectedRefs = useMemo(
  () => getSceneEntityRefsFromSelection(activeSelection),
```

```
[activeSelection],  
);
```

Avoid:

```
const selectedRefs = useDesignSceneStore((state) =>  
  getSceneEntityRefsFromSelection(state.designScene.activeSelection)  
);
```

Derived arrays/maps should generally be created with `useMemo` outside the store selector.

27. Naming rules

Use parent names for parent behavior:

```
SceneEntity  
SceneEntityRef  
SceneEntityBounds  
SceneEntityAlignmentGuide  
SceneEntityPlacementSurface  
SceneEntityMoveDragSurface  
SceneEntityRotationSurface
```

Do not use assembly-owned names for generic behavior:

```
AssemblyObjectAlignmentGuide  
AssemblyObjectAlignmentGuides  
AssemblyPlacementFootprint  
AssemblyPlacementElevationFrame  
AssemblyDragSurface
```

If a type or file is no longer assembly-specific, move or rename the real thing and update all imports directly.

28. No fake alias / bridge / compatibility rule

Do not add files whose only purpose is:

```
old-name re-export  
compatibility re-export  
legacy wrapper  
bridge from old folder to new folder
```

```
fake migration path  
temporary alias
```

Bad:

```
export * from "../new-location/realFile";
```

Preferred:

```
rename/move the real file  
update every import  
delete the old file  
run import checks
```

29. Red-flag searches

Before returning code, search for:

```
baseCenterPointInches  
placedAssemblies:  
designReservationZones:  
assembly-placement  
design-reservation-zone-placement  
assembly-move  
design-reservation-zone-move  
scene-entity-multi-move  
assembly-rotation  
design-reservation-zone-rotation  
AssemblyPlacementFeedback  
AssemblyPlacementInvalidReason  
applyAssemblyPlacementRules  
createAssemblyPlacementFeedback  
assemblyPlacementFeedback  
AssemblyObjectAlignmentGuide  
activeObjectAlignmentGuides  
AssemblyDragSurface  
DesignReservationZoneDragSurface  
SceneEntityMultiDragSurface  
AssemblyPlacementFeedbackLayer  
compat  
compatibility  
bridge  
alias
```

```
shim
legacy
migration
migrate
```

Not every match is automatically wrong, but every match must be reviewed.

30. Verification checklist

Minimum checks when only `src` is available:

```
TypeScript syntax/transpile check where possible
local import existence check
named import/export check
JSON parse check for catalog JSON
red-flag search
confirm no engine imports from features
confirm no fake alias/bridge files were added
confirm no duplicate selected scene-entity ownership reappeared
confirm AssemblyPrimitiveMesh remains visual-only
confirm AI panel remains local-message shell only
```

When the full project is available, run:

```
npm run lint
npm run build
npm run test
```

31. Manual regression checklist

After scene-entity refactor work, test:

1. Select assembly in perspective, floor plan, and elevation.
2. Confirm selected highlight appears correctly.
3. Select reservation zone in perspective, floor plan, and elevation.
4. Confirm selected highlight appears correctly.
5. Place assembly in floor plan.
6. Place reservation zone in floor plan.
7. Drag assembly in floor plan.
8. Drag reservation zone in floor plan.
9. Drag assembly in elevation.
10. Drag reservation zone in elevation.
11. Rotate assembly in floor plan.

12. Rotate reservation zone in floor plan.
13. Release rotation pointer outside object and confirm rotation finishes.
14. Multi-select assembly + assembly.
15. Multi-select reservation zone + reservation zone.
16. Multi-select assembly + reservation zone.
17. Confirm group guides and group movement.
18. Duplicate selected scene entities.
19. Delete selected scene entities.
20. Undo/redo placement, move, rotation, duplicate, and delete.
21. Confirm wall measurement guides still render.
22. Confirm alignment guides render in floor plan.
23. Confirm alignment guides render in elevation.
24. Confirm wall drawing still works only in floor plan.
25. Confirm wall segment editing still works.
26. Confirm wall elevation navigator still works.
27. Confirm sink/cooktop cutouts still derive from placed assemblies.
28. Confirm door/window wall openings still derive from placed assemblies.
29. Confirm AI chat does not mutate the scene.

32. Known follow-up behavior

The parent `SceneEntity` architecture is the source of truth going forward.

Known follow-up area:

elevation wall-face left/right guide-only alignment

Desired behavior:

when dragging a scene entity in elevation,
show visual guide lines when the object left/center/right aligns near the active
wall face left/center/right,
without forcing wall-edge snapping.

Do not implement aggressive wall snapping unless it is explicitly requested and behavior-tested.